

HANDS-ON LAB GUIDE

KUBERNETES

Storage

From an empty Ubuntu VM to a fully working PV → PVC
→ Pod chain over NFS, complete with ConfigMaps and
Secrets. A complete, from-scratch walkthrough.

NFS SERVER HOSTNAME	<code>nfs.it24.local</code>
NFS SERVER IP ADDRESS	<code>192.168.163.131</code>
NFS SERVER OS	<code>Ubuntu Server 26.04 LTS</code>
ADMIN USERNAME	<code>k8sadmin</code>
NFS EXPORT PATH	<code>/srv/nfs/k8s</code>
STORAGE CLASS	<code>nfs-client</code>
ESTIMATED DURATION	<code>90-120 minutes</code>

Persistent Volumes

PVCs

NFS

StorageClasses

ConfigMaps

Secrets

Table of Contents

00Lab Overview & Objectives

01Environment & Architecture

02Prerequisites

03Provisioning the NFS Server

Part A

04Preparing the Kubernetes Nodes

Part B

05Static Provisioning: PersistentVolume

Part C

06PersistentVolumeClaim & Binding

Part D

07Mounting Storage in a Pod

Part E

08Dynamic Provisioning (StorageClass)

Part F

09ConfigMaps

Part G

10Secrets

Part H

11Putting It All Together

Part I

12Cleanup

Part J

13Troubleshooting Reference

14Command Cheat Sheet & Appendix

● Server setup● Core storage objects● Workload & verification● Configuration● Sensitive data

Lab Overview & Objectives

This lab takes you from a completely empty virtual machine to a working, persistent, network-backed storage chain inside Kubernetes. You will provision a real NFS server from scratch, wire it into a Kubernetes cluster as a `PersistentVolume`, claim it with a `PersistentVolumeClaim`, mount it in a Pod, prove that data survives Pod deletion, then layer in dynamic provisioning, `ConfigMaps`, and `Secrets` — finishing with a single application manifest that uses all of it together.

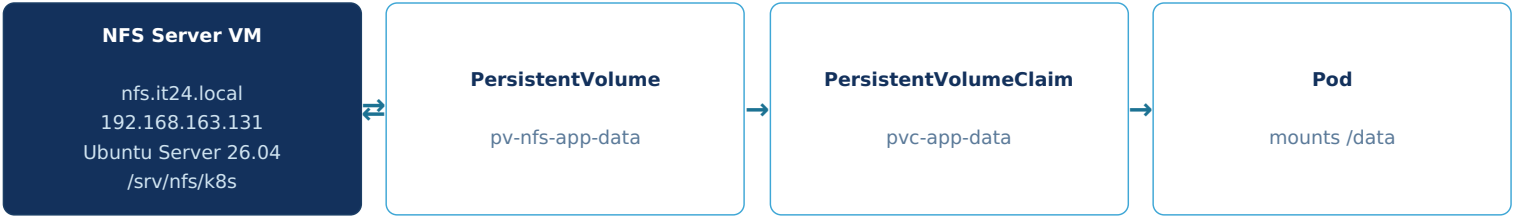
BY THE END OF THIS LAB YOU WILL BE ABLE TO:

- Install and configure an NFS server on Ubuntu Server, including exports and firewall rules
- Install the NFS client tooling required on every Kubernetes node
- Write and apply a static `PersistentVolume` backed by an NFS export
- Write and apply a `PersistentVolumeClaim` and confirm it binds correctly
- Mount a PVC inside a Pod and prove that data persists across Pod restarts
- Deploy an NFS-based dynamic provisioner and a `StorageClass` for on-demand storage
- Create and consume `ConfigMaps` as both environment variables and mounted files
- Create and consume `Secrets` safely, understanding the base64 caveat
- Combine a PVC, `ConfigMap`, and `Secret` into one realistic application deployment
- Tear everything down cleanly

i NOTE — Commands are written for Ubuntu Server on the NFS VM and for a Kubernetes cluster reachable via `kubect1` with cluster-admin rights (`kubeadm`, `k3s`, or `managed` — any distribution works as long as the nodes can route to 192.168.163.131).

Environment & Architecture

The lab environment consists of one dedicated NFS server VM and an existing Kubernetes cluster (control plane + one or more worker nodes) on the same network. The NFS server exports a directory over the network; Kubernetes nodes mount that export indirectly through the PV/PVC/Pod chain.



Component	Role	Details
NFS Server	Storage backend	Ubuntu 26.04, hostname <code>nfs.it24.local</code> , IP <code>192.168.163.131</code> , exports <code>/srv/nfs/k8s</code>
Kubernetes Nodes	NFS clients	Need <code>nfs-common</code> installed to mount NFSv4 shares
PersistentVolume	Cluster storage object	Points at the NFS server + export path
PersistentVolumeClaim	Storage request	Consumed by one or more Pods
StorageClass + Provisioner	Dynamic provisioning	Auto-creates PVs on demand from the same NFS export

✓ **TIP** — If your lab machines don't have working DNS, add a hosts entry on every Kubernetes node so `nfs.it24.local` resolves: `echo "192.168.163.131 nfs.it24.local nfs.it24" | sudo tee -a /etc/hosts .`

Prerequisites

- A fresh Ubuntu Server 26.04 VM for the NFS server (2 vCPU / 2 GB RAM / 20 GB disk is plenty), with network access to the Kubernetes nodes
- An existing, working Kubernetes cluster (v1.28+) with `kubectl` configured and cluster-admin access
- Root or sudo access on the NFS VM and on every Kubernetes node
- `helm` installed locally if you plan to deploy the dynamic NFS provisioner via Helm in Part F (a manual manifest alternative is also provided)
- All machines on the same routable network (this guide assumes the `192.168.163.0/24` subnet)

Provisioning the NFS Server (nfs.it24.local)

We start from a completely empty Ubuntu Server 26.04 installation and build the NFS server step by step.

1 Install Ubuntu Server 26.04

Boot the Ubuntu Server 26.04 ISO on the VM and walk through the standard installer. When prompted:

- Hostname: `nfs.it24.local`
- Username: `k8sadmin` (administrative user, sudo enabled)
- Password: set per your lab's credential policy — used only for local/sudo login on this VM
- Disk: use the entire disk with default LVM partitioning
- Skip additional snaps (server minimal install is sufficient)

Reboot into the installed system once setup completes, then log in as `k8sadmin`.

2 Confirm / set the hostname

```
***
sudo hostnamectl set-hostname nfs.it24.local
hostnamectl status
```

Edit `/etc/hosts` so the hostname resolves locally as well:

```
***
sudo sed -i "s/^127.0.1.1.*/127.0.1.1 nfs.it24.local nfs.it24/" /etc/hosts
cat /etc/hosts
```

3 Configure a static IP address (192.168.163.131)

Identify the interface name, then edit the Netplan config generated by the installer:

```
***
ip -brief link show
sudo nano /etc/netplan/00-installer-config.yaml
```

```
***
network:
  version: 2
  ethernets:
    ens33:
      dhcp4: false
      match:
        macaddress: "00:0c:29:3a:9b:7e"
      set-name: ens33
      addresses: [192.168.163.131/24]
      routes:
        - to: default
          via: 192.168.163.1
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
```

Replace `00:0c:29:3a:9b:7e` with your VM's actual MAC address (find it with `ip link show ens33`) — the optional `match` block pins this config to that specific NIC even if device naming changes on a future reboot.

Apply and verify:

```
***
sudo netplan apply
ip addr show ens33 | grep 192.168.163.131
```

4

Update the system and install the NFS server package

```
...
sudo apt update && sudo apt -y upgrade
sudo apt -y install nfs-kernel-server
```

Confirm the service is enabled and running:

```
...
sudo systemctl enable --now nfs-kernel-server
sudo systemctl status nfs-kernel-server --no-pager
```

5

Create the export directory

```
...
sudo mkdir -p /srv/nfs/k8s
sudo chown -R nobody:nogroup /srv/nfs/k8s
sudo chmod 0777 /srv/nfs/k8s
```

NOTE — `0777` keeps the lab friction-free across arbitrary container UIDs. In production, scope permissions to the specific UID/GID your workloads run as, or use per-directory export mapping.

6

Define the export in /etc/exports

```
...
sudo nano /etc/exports
```

Add a line granting the Kubernetes subnet read/write, no-root-squash access:

```
...
/srv/nfs/k8s 192.168.163.0/24(rw, sync, no_subtree_check, no_root_squash)
```

Apply the export table and verify:

```
...
sudo exportfs -ra
sudo exportfs -v
```

Expected output:

```
...
/srv/nfs/k8s
192.168.163.0/24(rw, wdelay, no_root_squash, no_subtree_check, sec=sys, rw, secure, no_root_squash, no_all_squash)
```

7

Open the firewall (if UFW is active)

```
...
sudo ufw allow from 192.168.163.0/24 to any port nfs
sudo ufw allow from 192.168.163.0/24 to any port 2049
sudo ufw allow from 192.168.163.0/24 to any port 111
sudo ufw status
```

8

Restart NFS services and confirm they're listening

```
...
sudo systemctl restart nfs-kernel-server
sudo rpcinfo -p localhost | grep nfs
ss -tulpn | grep -E '2049|111'
```



CHECKPOINT — The NFS server is ready when `exportfs -v` lists `/srv/nfs/k8s` and `rpcinfo -p localhost` shows the `nfs` program listening on port 2049.

Preparing the Kubernetes Nodes

Every node that might schedule a Pod using the NFS-backed volume needs the NFS client utilities installed — the kubelet delegates the actual mount to the host's `mount.nfs` binary.

1 Install nfs-common on every node

Run this on the control-plane node and every worker node:

```
...
sudo apt update
sudo apt -y install nfs-common
```

2 (Optional but recommended) Add a hosts entry for nfs.it24.local

```
...
echo "192.168.163.131 nfs.it24.local nfs.it24" | sudo tee -a /etc/hosts
```

3 Manually test the mount from one node before touching Kubernetes

```
...
sudo mkdir -p /mnt/nfs-test
sudo mount -t nfs 192.168.163.131:/srv/nfs/k8s /mnt/nfs-test
echo "hello from $(hostname)" | sudo tee /mnt/nfs-test/test.txt
cat /mnt/nfs-test/test.txt
sudo umount /mnt/nfs-test
```

! IF THE MOUNT HANGS — Double-check the UFW rules on the NFS server (Part A, step 7) and that the node can reach port 2049: `nc -zv 192.168.163.131 2049`.

Static Provisioning: PersistentVolume

With NFS reachable from every node, define a `PersistentVolume` that represents the export as a cluster storage resource.

```
...
cat <<'EOF' > pv-nfs-app-data.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-nfs-app-data
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Retain
  nfs:
    path: /srv/nfs/k8s
    server: 192.168.163.131
EOF
kubectl apply -f pv-nfs-app-data.yaml
```

Verify:

```
...
kubectl get pv pv-nfs-app-data
kubectl describe pv pv-nfs-app-data
```

The `STATUS` column should read `Available` — it has no claim bound to it yet.

Field	Meaning
<code>capacity.storage</code>	Nominal size Kubernetes tracks; NFS itself does not enforce a quota unless you configure one separately
<code>accessModes</code>	<code>ReadWriteMany</code> — NFS supports concurrent mounts from multiple nodes
<code>persistentVolumeReclaimPolicy</code>	<code>Retain</code> keeps the data after the PVC is deleted, requiring manual cleanup — safest choice for a lab
<code>nfs.server</code> / <code>nfs.path</code>	Exactly what we configured in Part A

PersistentVolumeClaim & Binding

Now request storage from the application side with a `PersistentVolumeClaim` .

```
***
cat <<'EOF' > pvc-app-data.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-app-data
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 5Gi
  storageClassName: ""
  volumeName: pv-nfs-app-data
EOF
kubectl apply -f pvc-app-data.yaml
```



NOTE — Setting `storageClassName: ""` and an explicit `volumeName` forces this PVC to bind to our specific static PV rather than triggering dynamic provisioning (which we set up separately in Part F).

Confirm binding:

```
***
kubectl get pvc pvc-app-data
kubectl get pv pv-nfs-app-data
```

Both objects should now show `STATUS: Bound` , and `kubectl get pvc` should list `pv-nfs-app-data` under the `VOLUME` column.

Mounting Storage in a Pod & Testing Persistence

► Deploy a test Pod

```

cat <<'EOF' > pod-storage-test.yaml
apiVersion: v1
kind: Pod
metadata:
  name: storage-test
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "sleep 3600"]
      volumeMounts:
        - name: data
          mountPath: /data
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: pvc-app-data
EOF
kubectl apply -f pod-storage-test.yaml
kubectl wait --for=condition=Ready pod/storage-test --timeout=60s

```

► Write data, delete the Pod, and prove it survives

1 Write a file from inside the Pod

```

cat <<'EOF' >
kubectl exec storage-test -- sh -c 'echo "persisted at $(date)" > /data/proof.txt'
kubectl exec storage-test -- cat /data/proof.txt
EOF

```

2 Confirm the file landed on the NFS server itself

```

cat <<'EOF' >
# run on nfs.it24.local
cat /srv/nfs/k8s/proof.txt
EOF

```

3 Delete the Pod entirely

```

cat <<'EOF' >
kubectl delete pod storage-test
EOF

```

4 Recreate it and check the data is still there

```

cat <<'EOF' >
kubectl apply -f pod-storage-test.yaml
kubectl wait --for=condition=Ready pod/storage-test --timeout=60s
kubectl exec storage-test -- cat /data/proof.txt
EOF

```



CHECKPOINT — If `proof.txt` still shows the original timestamp after the Pod was fully deleted and recreated, you've proven persistence end-to-end: Pod → PVC → PV → NFS export.

Dynamic Provisioning with a StorageClass

Static PVs work, but don't scale to dozens of teams requesting storage independently. This part deploys the `nfs-subdir-external-provisioner`, which watches for PVCs referencing a given StorageClass and automatically creates a subdirectory (and matching PV) on our NFS export for each one.

► Option 1 — Install via Helm

```
...
helm repo add nfs-subdir-external-provisioner \
  https://kubernetes-sigs.github.io/nfs-subdir-external-provisioner
helm repo update

helm install nfs-provisioner nfs-subdir-external-provisioner/nfs-subdir-external-provisioner \
  --set nfs.server=192.168.163.131 \
  --set nfs.path=/srv/nfs/k8s \
  --set storageClass.name=nfs-client \
  --set storageClass.defaultClass=false
```

► Option 2 — Plain manifest (no Helm)

```
...
cat <<'EOF' > nfs-storageclass.yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: nfs-client
provisioner: cluster.local/nfs-subdir-external-provisioner
parameters:
  archiveOnDelete: "false"
reclaimPolicy: Delete
volumeBindingMode: Immediate
EOF
kubectl apply -f nfs-storageclass.yaml
```

The provisioner Deployment itself (RBAC, ServiceAccount, and the container that talks to NFS) follows the upstream `nfs-subdir-external-provisioner` manifests — the Helm chart in Option 1 creates all of it for you in one command, which is why it's the recommended path for this lab.

► Request storage dynamically

```
...
cat <<'EOF' > pvc-dynamic.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-dynamic-demo
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 1Gi
  storageClassName: nfs-client
EOF
kubectl apply -f pvc-dynamic.yaml
kubectl get pvc pvc-dynamic-demo
kubectl get pv | grep pvc-dynamic-demo
```



CHECKPOINT — No PV was written by hand this time — the provisioner created one automatically, plus a matching subdirectory under `/srv/nfs/k8s` on `nfs.it24.local`. Confirm with `ls /srv/nfs/k8s` on the NFS server.

ConfigMaps

A ConfigMap stores non-sensitive configuration — plain text key-value pairs — separately from your container image, so the same image can run in dev, staging, and production with different settings.

i TWO WAYS TO CONSUME A CONFIGMAP — A Pod can read a ConfigMap's keys as **environment variables** (good for simple settings like `LOG_LEVEL`), or as **mounted files** where every key becomes a file on disk (good for whole config files like `nginx.conf`). This lab demonstrates both, in the same Pod, so you can see exactly how each one looks from inside the container.

1

Create the ConfigMap

This ConfigMap has two simple keys (`APP_ENV`, `LOG_LEVEL`) and one file-style key (`app.properties`) holding multi-line text.

```
***
cat <<'EOF' > configmap-app.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: "production"
  LOG_LEVEL: "info"
  app.properties: |
    storage.mountPath=/data
    storage.server=192.168.163.131
EOF
kubectl apply -f configmap-app.yaml
```

Confirm the three keys are stored:

```
***
kubectl get configmap app-config -o yaml
```

2

Deploy a Pod that consumes it both ways

This single Pod reads `APP_ENV` and `LOG_LEVEL` as environment variables via `envFrom`, **and** mounts the whole ConfigMap as files under `/etc/app-config/` via a volume — so you can compare both side by side.

```
***
cat <<'EOF' > pod-configmap-demo.yaml
apiVersion: v1
kind: Pod
metadata:
  name: configmap-demo
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "sleep 3600"]
      envFrom:
        - configMapRef:
            name: app-config
      volumeMounts:
        - name: config-vol
          mountPath: /etc/app-config
  volumes:
    - name: config-vol
      configMap:
        name: app-config
EOF
kubectl apply -f pod-configmap-demo.yaml
kubectl wait --for=condition=Ready pod/configmap-demo --timeout=60s
```

3

Verify the environment variables

```
...
kubectl exec configmap-demo -- env | grep -E 'APP_ENV|LOG_LEVEL'
```

Expected output:

```
...
APP_ENV=production
LOG_LEVEL=info
```

4

Verify the mounted files

```
...
kubectl exec configmap-demo -- ls -la /etc/app-config
kubectl exec configmap-demo -- cat /etc/app-config/app.properties
```

Expected output — every key in `data` became its own file in the mount directory:

```
...
APP_ENV LOG_LEVEL app.properties

storage.mountPath=/data
storage.server=192.168.163.131
```

Consumption method	What ends up in the container	Best for
<code>envFrom</code> / <code>env.valueFrom</code>	Individual environment variables, e.g. <code>\$APP_ENV</code>	Simple flags, ports, feature toggles
<code>volumes.configMap</code>	One file per key, e.g. <code>/etc/app-config/app.properties</code>	Whole config files (nginx.conf, app.yaml, etc.)

✓

CHECKPOINT — If both commands in steps 3 and 4 returned the values above, the ConfigMap is correctly wired into the Pod through both paths.

Secrets

A Secret works exactly like a ConfigMap — same two consumption methods, same API shape — but it's meant for sensitive values (passwords, tokens, keys) and Kubernetes stores/transmits it with tighter access control. The one real difference you'll notice: values under `data` must be base64-encoded.

1

Encode the values (or let kubectl do it for you)

Base64-encode each value by hand:

```
...
echo -n 'admin' | base64          # -> YWRtaW4=
echo -n 'S#uperSecret!' | base64 # -> UyN1cGVyU2VjcmV0IQ==
```

...or skip the encoding step entirely and let `kubectl` handle it — this is the recommended, less error-prone approach:

```
...
kubectl create secret generic db-credentials \
  --from-literal=username=admin \
  --from-literal=password='S#uperSecret!'
```

2

Or define it declaratively as YAML

Equivalent to the `kubectl create secret` command above — useful when you want the Secret checked into a manifest repo (with the actual values injected securely at deploy time, not committed in plain text):

```
...
cat <<'EOF' > secret-db.yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
type: Opaque
data:
  username: YWRtaW4=
  password: UyN1cGVyU2VjcmV0IQ==
EOF
kubectl apply -f secret-db.yaml
```

Confirm it exists (note that `kubectl get` never prints the decoded values):

```
...
kubectl get secret db-credentials
```

3 Deploy a Pod that consumes it both ways

Just like the ConfigMap demo — one Pod, both methods side by side, so you can see exactly how each one surfaces inside the container.

```
cat <<'EOF' > pod-secret-demo.yaml
apiVersion: v1
kind: Pod
metadata:
  name: secret-demo
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "sleep 3600"]
      env:
        - name: DB_USERNAME
          valueFrom:
            secretKeyRef:
              name: db-credentials
              key: username
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: db-credentials
              key: password
      volumeMounts:
        - name: db-secret-vol
          mountPath: /etc/db-secret
          readOnly: true
  volumes:
    - name: db-secret-vol
      secret:
        secretName: db-credentials
EOF
kubectl apply -f pod-secret-demo.yaml
kubectl wait --for=condition=Ready pod/secret-demo --timeout=60s
```

4 Verify the environment variables

```
cat <<'EOF' > verify-env.sh
kubectl exec secret-demo -- env | grep -E 'DB_USERNAME|DB_PASSWORD'
```

Expected output — Kubernetes decoded the base64 for you automatically before injecting it:

```
cat <<'EOF' > verify-env-output.txt
DB_USERNAME=admin
DB_PASSWORD=$superSecret!
```

5 Verify the mounted files

```
cat <<'EOF' > verify-mount.sh
kubectl exec secret-demo -- ls -la /etc/db-secret
kubectl exec secret-demo -- cat /etc/db-secret/username
```

Expected output — again, one file per key, already decoded to plain text on disk:

```
cat <<'EOF' > verify-mount-output.txt
username  password

admin
```

Consumption method	What ends up in the container	Best for
<code>env.valueFrom.secretKeyRef</code>	Individual environment variables, e.g. <code>\$DB_PASSWORD</code>	Credentials an app reads from its environment
<code>volumes.secret</code>	One file per key under the mount path, plain text	TLS certs, SSH keys, files an app expects to <code>open()</code>

❗ **BASE64 CAUTION** — Base64 is an encoding, not encryption. Anyone with read access to the Secret object via the API can decode it in one command:

```
****
kubectl get secret db-credentials -o jsonpath='{.data.password}' | base64 -d
```

Restrict access with RBAC, enable encryption at rest on the API server, and consider an external secrets manager (Vault, Sealed Secrets, cloud KMS integrations) for anything beyond a lab.

✅ **CHECKPOINT** — If steps 4 and 5 both returned the expected values, the Secret is correctly wired into the Pod through both paths — and you've directly observed that Kubernetes decodes base64 for you at consumption time, it just never stores or transmits it as plain text.

Putting It All Together: A Full Application

This final manifest combines everything from the lab into one realistic Deployment: persistent NFS-backed storage via the dynamic `nfs-client` StorageClass, configuration via ConfigMap, and credentials via Secret.

```
...
cat <<'EOF' > app-full.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: app-pvc
spec:
  accessModes: ["ReadWriteMany"]
  storageClassName: nfs-client
  resources:
    requests:
      storage: 2Gi
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: demo-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: demo-app
  template:
    metadata:
      labels:
        app: demo-app
    spec:
      containers:
        - name: app
          image: busybox:1.36
          command: ["sh", "-c", "sleep 86400"]
          envFrom:
            - configMapRef:
                name: app-config
          env:
            - name: DB_USERNAME
              valueFrom:
                secretKeyRef:
                  name: db-credentials
                  key: username
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: db-credentials
                  key: password
      volumeMounts:
        - name: data
          mountPath: /data
        - name: config-vol
          mountPath: /etc/app-config
        - name: db-secret-vol
          mountPath: /etc/db-secret
          readOnly: true
      volumes:
        - name: data
          persistentVolumeClaim:
            claimName: app-pvc
        - name: config-vol
          configMap:
            name: app-config
        - name: db-secret-vol
          secret:
            secretName: db-credentials
EOF
kubectl apply -f app-full.yaml
kubectl rollout status deployment/demo-app
```

► Verify everything is wired correctly

```
...
kubectl get pods -l app=demo-app
POD=$(kubectl get pod -l app=demo-app -o jsonpath='{.items[0].metadata.name}')
kubectl exec $POD -- env | grep -E 'APP_ENV|LOG_LEVEL|DB_'
kubectl exec $POD -- cat /etc/app-config/app.properties
kubectl exec $POD -- ls -la /etc/db-secret
kubectl exec $POD -- sh -c 'echo "written by demo-app" > /data/from-deployment.txt'
kubectl exec $POD -- cat /data/from-deployment.txt
```



CHECKPOINT — Both replicas of `demo-app` share the same NFS-backed PVC (ReadWriteMany), read config from the mounted ConfigMap file, and receive DB credentials from the Secret via environment variables — the complete storage + config + secrets picture in one Deployment.

Cleanup

Tear down lab resources in the reverse order they were created, so nothing is left orphaned.

```
...
kubectl delete -f app-full.yaml
kubectl delete -f secret-db.yaml
kubectl delete -f configmap-app.yaml
kubectl delete -f pvc-dynamic.yaml
kubectl delete -f nfs-storageclass.yaml      # skip if installed via Helm
helm uninstall nfs-provisioner               # if Helm was used
kubectl delete -f pod-storage-test.yaml
kubectl delete -f pvc-app-data.yaml
kubectl delete -f pv-nfs-app-data.yaml
```

On the NFS server, remove the export and data once you're done (optional — skip if you plan to reuse the server):

```
...
# on nfs.it24.local
sudo rm -rf /srv/nfs/k8s/*
sudo sed -i 's|/srv/nfs/k8s|d|' /etc/exports
sudo exportfs -ra
```

Troubleshooting Reference

Symptom	Likely Cause	Fix
PVC stuck in <code>Pending</code>	No matching PV, or StorageClass provisioner not running	<code>kubectl describe pvc <name></code> for events; check <code>kubectl get pods -n default -l app=nfs-subdir-external-provisioner</code>
Pod stuck in <code>ContainerCreating</code>	NFS mount failing from the node	Check <code>kubectl describe pod</code> events; test <code>mount -t nfs 192.168.163.131:/srv/nfs/k8s /mnt/test</code> manually on the node
<code>mount.nfs:</code> <code>Connection timed out</code>	Firewall blocking port 2049/111, or wrong IP/hostname	Re-check UFW rules on nfs.it24.local; <code>nc -zv 192.168.163.131 2049</code>
<code>Permission denied</code> writing to mount	Export permissions or <code>root_squash</code> mismatch	Confirm <code>no_root_squash</code> in <code>/etc/exports</code> and directory mode <code>0777</code> (lab) or matching UID/GID (production)
Dynamic PVC never provisions	Provisioner Deployment CrashLooping or wrong <code>provisioner</code> name	<code>kubectl logs deploy/nfs-provisioner-nfs-subdir-external-provisioner</code> ; confirm StorageClass <code>provisioner</code> matches exactly
Secret value looks wrong in the Pod	Value wasn't base64-encoded, or was double-encoded	Re-create with <code>kubectl create secret generic ... --from-literal=</code> to avoid manual encoding mistakes
<code>nfs.it24.local</code> doesn't resolve	No DNS entry for the lab hostname	Add <code>192.168.163.131 nfs.it24.local</code> to <code>/etc/hosts</code> on every node, or use the IP directly in the PV

Command Cheat Sheet & Appendix

► NFS server (nfs.it24.local)

```
...
sudo systemctl status nfs-kernel-server --no-pager
sudo exportfs -v
sudo exportfs -ra
sudo showmount -e 192.168.163.131
```

► Kubernetes storage objects

```
...
kubectl get pv
kubectl get pvc
kubectl describe pv <name>
kubectl describe pvc <name>
kubectl get storageclass
kubectl get sc nfs-client -o yaml
```

► ConfigMaps & Secrets

```
...
kubectl create configmap app-config --from-literal=APP_ENV=production
kubectl get configmap app-config -o yaml
kubectl create secret generic db-credentials --from-literal=username=admin
kubectl get secret db-credentials -o jsonpath='{.data.username}' | base64 -d
```

► Quick reference: object relationships

Object	Consumes	Consumed by
PersistentVolume	NFS export (or other backend)	PersistentVolumeClaim
PersistentVolumeClaim	PersistentVolume (static or dynamic)	Pod / Deployment
StorageClass	CSI driver / external provisioner	PersistentVolumeClaim
ConfigMap	Plain key-value / file data	Pod (env or volume)
Secret	Base64-encoded key-value data	Pod (env or volume)